Skill Builder Feedback Report

Position: Skill Assessment - AI Engineer
Date: 8.7.2026
Session ID: dlai_interview_0707712759b048389d962858257fac61

📊 Skill Builder Overview

SUMMARY

You discussed your advanced multi-agent coding framework, miniSashimi, which orchestrates 13 specialized roles across four development phases. You also detailed your experience fine-tuning a local Gemma model using LoRA and your plans for state management and self-healing runtime environments.

STANDOUT STRENGTHS

Your system architecture design is highly sophisticated, particularly your multi-tiered routing logic and the implementation of a Human-in-the-Loop fallback to prevent infinite self-healing loops. You also demonstrated strong practical skills in local model deployment and performance optimization using Ollama and LM Studio.

KEY DEVELOPMENT AREAS

Integrating automated testing directly into your verification phase to catch runtime errors before LLM reviews, and implementing a hybrid memory architecture that combines chronological state tracking with vector-based retrieval.

RECOMMENDED FOCUS

Prioritize building the automated testing integration using a sandboxed execution environment paired with a test runner like pytest to establish

a robust self-healing loop.

Recommended Learning Path

Curated based on your skill gap analysis

Long-Term Agentic Memory with LangGraph

Short Course

This course directly addresses your goal of building a robust, stateful memory system. It will help you implement the hybrid chronological and semantic memory architecture you discussed using LangGraph's built-in state management.

AI Agents in LangGraph **Short Course**

Since you are building a highly complex 13-role multi-agent system, this course will teach you how to transition from custom JSON-based orchestration to a robust, graph-based agent framework designed for complex state transitions and routing.

Fine-tuning and Reinforcement Learning for LLMs: Intro to Post-Training

Long Course

This course will help you deepen your fine-tuning skills beyond basic LoRA. It covers post-training lifecycles, evaluation, error analysis, and reinforcement learning techniques to make your local CodeMonkey model more reliable.

🎯 Skill Assessments

Multi-Agent Orchestration & Routing

Key Evidence:

> You designed miniSashimi, a multi-phase framework spanning Planning, Worker, Review, and Runtime levels with 13 configurable model roles.

> You implemented an intelligent provider routing system supporting native APIs, OpenRouter fallbacks, and local instances like LM Studio and Ollama.

> You established a Human-in-the-Loop double-bottom pattern to halt autonomous self-healing loops after a set threshold and prompt the user.

> **Path to Next Level:**
> To enhance your orchestration, consider adopting structured frameworks like LangGraph or AutoGen to manage complex state transitions and conditional routing more robustly than custom JSON state files. Implement strict schema validation for agent handoffs using Pydantic to prevent parsing failures between phases. You can also explore the Agent Communication Protocol to standardize how your local and cloud-based agents exchange messages.

Local LLM Fine-Tuning & Optimization

Key Evidence:

> You fine-tuned a Gemma model using LoRA to act as a local CodeMonkey to reduce API token usage.

> You generated synthetic training data using a two-step LLM pipeline to create structured coding questions and answers.

> You deployed and benchmarked the model locally using Ollama and LM Studio, tracking prompt processing and token generation speeds.

> **Path to Next Level:**

Path to Next Level:

To improve your fine-tuning pipeline, implement systematic evaluation of your fine-tuned model using LLM-as-a-judge frameworks or automated code-generation benchmarks like HumanEval. Explore quantization techniques such as AWQ or GGUF to further optimize local inference speed and memory footprint on your hardware. Additionally, look into post-training alignment techniques like Direct Preference Optimization to refine the model's adherence to specific coding styles.

State Management & Vector Databases

Key Evidence:

> You manage global state by having each agent report back to an orchestrator that maintains a shortened JSON summary.

> You proposed integrating LanceDB as a temporary vector database to store agent feedback as embeddings and reduce context window usage.

> You designed a hybrid memory approach to preserve chronological order by keeping recent steps in full text while offloading older history to RAG.

Path to Next Level:

Implement your hybrid memory design by using a sliding window for the active conversation context alongside LanceDB for semantic retrieval. Use metadata filtering in LanceDB to tag embeddings with timestamps or phase identifiers, allowing your agents to query both semantically and chronologically. Explore semantic caching to store and reuse frequent agent queries, reducing redundant embedding computations and latency.